AI-POWERED DEFECT RECURRENCE DETECTION SYSTEM

User enters new defect → System compares against historical repair data → Returns:

- ***Whether defect is recurring.***
- ***When it occurred.***
- ***Details of past events.***
- ***Likely solutions.***.

System Overview:

The QE Defect AI is a desktop application that uses machine learning to help quality engineers identify recurring defects by analyzing historical repair records. When a user enters a new defect description, the system compares it against historical data and returns:

- Whether the defect is recurring
- When it previously occurred
- Details of past events
- Likely solutions based on historical fixes

Tab 1: New Defect Entry

- **Input Fields:**

Field	Type	Description
Defect Description	Multi-line Text Box	The defect text to analyze
Plant Section	Dropdown	Filter by specific section
Date	Date Picker	When defect occurred
Severity	Optional Dropdown	Low/Medium/High

- **Section Dropdown Options:**
  - 341 – Internal Assembly
  - 361 – Paint Line
  - 333 – Core Coil Build
  - All Sections (default)
- **Output Panel:**

    - Recurrence Status (YES/NO with color coding)
    - Similarity Score (0.00 - 1.00)
    - Table of Past Occurrences with similarity scores
    - Suggested Corrective Action with confidence percentage

Tab 2: Repair Record Import

- Features:
    - Upload Excel file
    - Auto-detect column headers
    - Interactive column mapping wizard:
        - Description column
        - Date column
        - Section column
        - Resolution column
        - Category/Sub-category columns (optional)
    - Save mapping configuration
    - Category/Sub-category columns (optional)
- **Tab 2: Repair Record Import**
    - Visualizations:
        - Pareto Chart - Top 10 recurring defects
        - Section Heatmap - Defect density by area
        - Time Series - Defect frequency over time
        - Category Distribution - Pie/bar charts
    - Filters:
        - Time window (30/90/365 days)
        - Section filter
        - Category/Sub-category filter

- **Likely Implementation Concept**:
    - **Steps**
        - Step 1 – Convert descriptions into embeddings.
        - Step 2 – When user enters new defect.
        - Step 3 – Compare with cosine similarity (If similarity > threshold (e.g., 0.75) → mark as recurring.)
    - **Determining "Likely Solution"**
        - Once similar records are found:

        - Pull their resolution column
        - Rank by frequency
        - Return most common fix
    - Example:
      If 7 similar past defects:
        - 5 times → "Adjust torque setting"
        - 2 times → "Replace hinge bracket"
    - Return:
      “Most likely solution: Adjust torque setting (71% historical success)”

<u>Likely Core Feature</u>:

GUI:

- PyQt6 (recommended)
- OR Tkinter (simpler)

Data:

- pandas
- openpyxl

NLP:

- sentence-transformers
- scikit-learn

Optional features to add (Advanced):

- FAISS (fast similarity search for large datasets)
- SQLite (store historical embeddings)
- Pareto Chart Generator
  Show top 10 recurring defects in:
    - 341 Internal Assembly
    - 361 Paint Line
    - 333 Core Coil Build
- Section Level Heatmap.

- ## <u>Likely Implementation Concept</u>:

```
QE_Defect_AI/
|
├── main_window.py          # Application entry point
├── build_exe.bat           # PyInstaller build script
|
├── core/                   # Backend logic
|   ├── __init__.py
|   ├── similarity_engine.py   # ML embedding & comparison
|   ├── cache.py              # Embedding cache management
|   ├── data_cache.py         # DataFrame snapshot cache
|   ├── data_loader.py        # CSV/Excel loading
|   ├── reporting.py          # PDF report generation
|   └── path_utils.py         # Cross-platform path handling
|
├── gui/                    # Frontend interface
|   ├── __init__.py
|   ├── main_window.py        # Main window with tabs
|   ├── defect_tab.py         # New defect entry tab
|   ├── import_tab.py         # Data import tab
|   ├── trends_tab.py         # Analytics tab
|   ├── about_dialog.py       # About dialog
|   └── theme_manager.py      # Dark/light theme switching
|
├── data/                   # Default data files
|   └── AI_Ready_Defect_Recurrence_Master.csv
|
├── reports/                # Generated PDF reports
|
└── installer/              # Inno Setup output
    └── QE_Defect_AI_Setup.exe
```

CACHING STRATEGY

- **Two-Layer Cache System:**
  - DataFrame Snapshot (*.pkl)
  - Stores cleaned/parsed DataFrame
  - Validated by file modification time and size
  - Fast loading without reparsing CSV
- **Embedding Cache (*.npz + *.json)**
  - Stores numpy array of embeddings
  - Metadata includes row count, file hash, model name
  - Automatically invalidated when data changes

MACHINE LEARNING COMPONENTS

- **Model: all-MiniLM-L6-v2**
  - Type: Transformer-based sentence embedding model

- Output: 384-dimensional vectors
- Training: Pre-trained on 1B+ sentence pairs
- Speed: ~10,000 sentences/second on CPU

- **Threshold Configuration**
  - Default: 0.75
  - Adjustable per user preference
  - Higher = fewer but more accurate matches
  - Lower = more matches but potentially less relevant

INSTALLATION

- Required Packages:
  "pip install pyqt6 pandas openpyxl sentence-transformers scikit-learn matplotlib numpy"

Package	Purpose
PyQt6	Graphical user interface
pandas	Data manipulation and analysis
openpyxl	Excel file support
sentence-transformers	AI/ML embedding model
scikit-learn	Cosine similarity calculations
matplotlib	Charts and visualizations
numpy	Numerical array operations

TESTING FOR FUNCTIONALITY

HOW TO CONVERT TO .EXE

- Step 0: Run the *******_**Setup.exe** downloaded.

<u>The steps below are for if for some reason the setup failed.</u>

- Step 2: Install PyInstaller

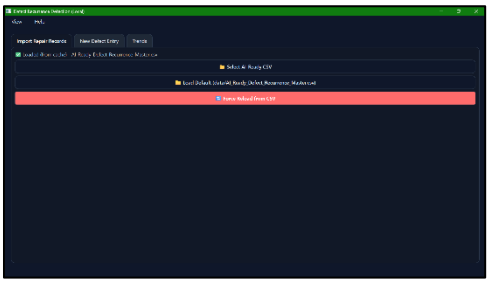
  <u>"pip install pyinstaller"</u>

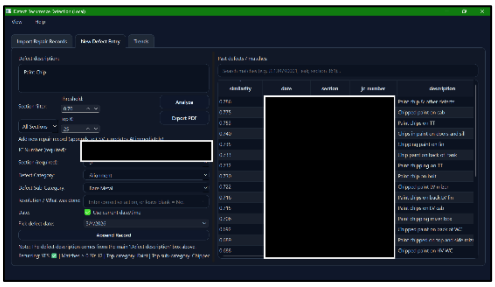
- Step 3: Create a Build Script

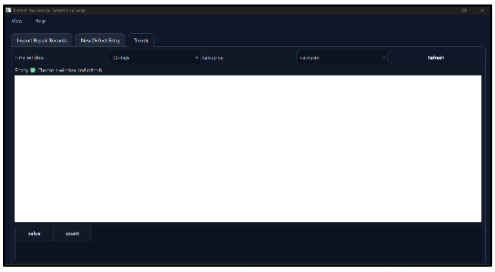
  Create build_exe.bat in your project root (COPY THE CURRENT BULD_EXE.BAT FILE)

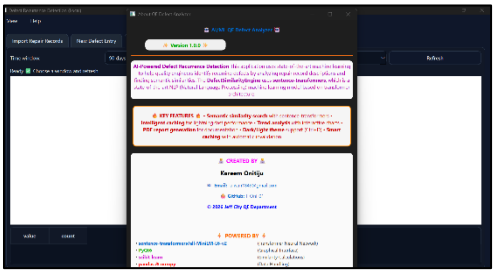
- Step 4: Handle File Paths for EXE by creating a "path_utils.py" in the core folder
- Step 6: Create an Icon (Optional) and put it in the project root folder.
- Step 7: Run the Build.

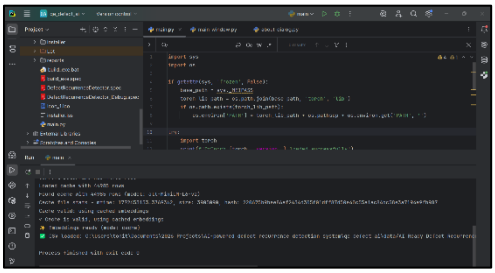
  <u>"build_exe.bat"</u>
- Step 8: Test the Executable (should be in the dist folder).
- Step 10: Create an Installer with Inno Setup (Optional)
  - Download Inno Setup and create installer.iss:
- **<u>Images</u>**

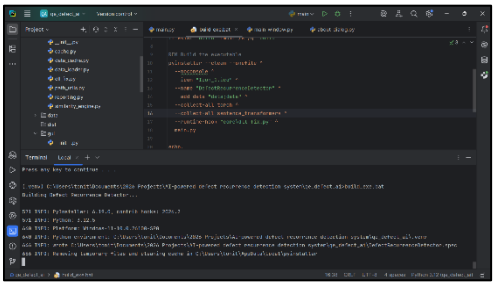












- **Conclusion**

  The QE Defect AI application successfully combines modern machine learning techniques with an intuitive graphical interface to solve real-world quality engineering problems. By leveraging sentence transformers for semantic understanding, the system can identify recurring defects based on meaning rather than simple keyword matching, providing more accurate and useful results for quality engineers.